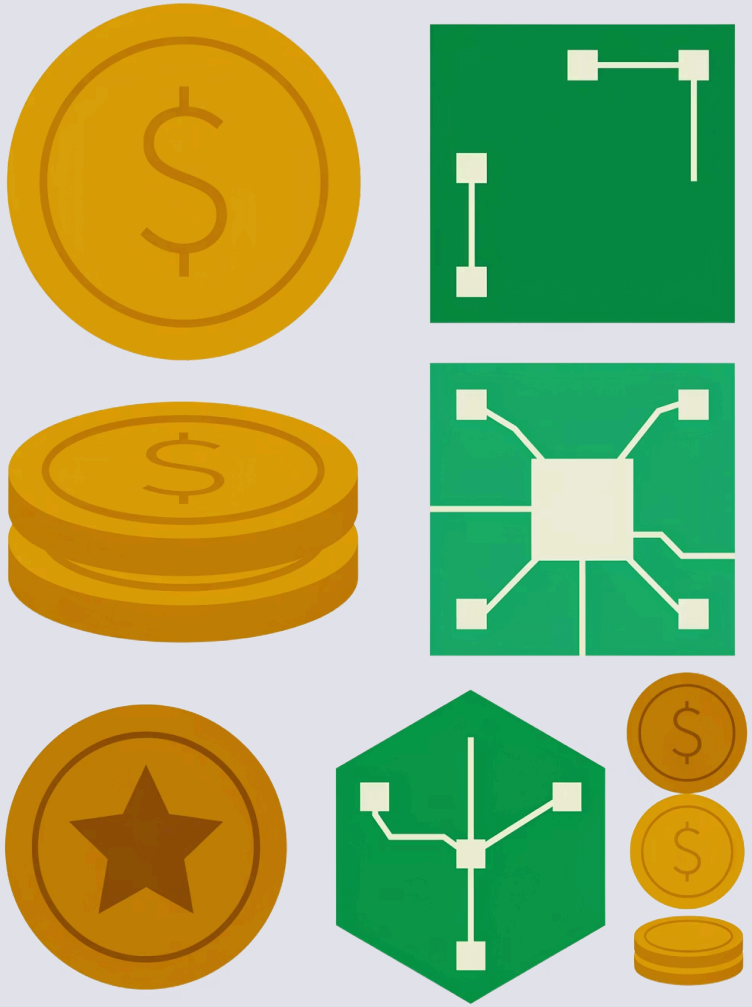


Token Economics

How to estimate AI costs and prevent runaway bills

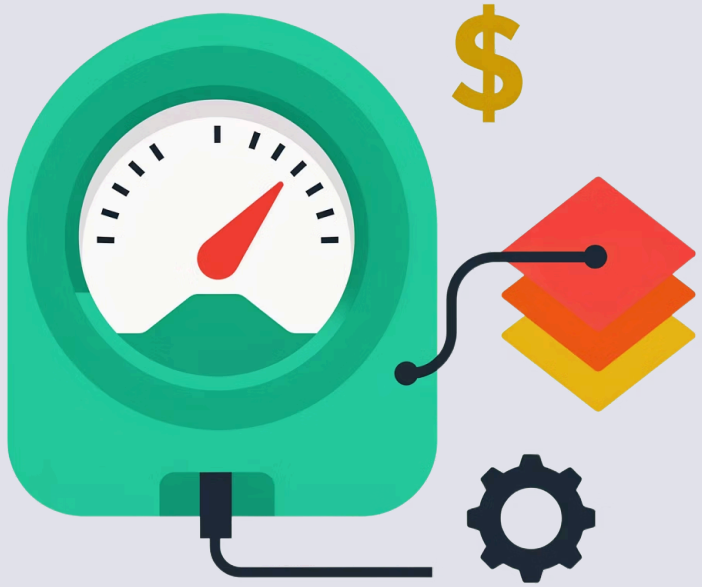
Introduction to Generative AI

You Pay Per Token, Not Per Request

When you call an LLM API (Claude, GPT, etc.), you are not billed per request

You are billed per token

Understanding tokens, estimating costs, and preventing runaway bills is a core skill for anyone building AI-powered apps



What Is a Token?

The smallest unit of text a language model reads or writes. Roughly:

~4 characters of English text

~¾ of a word on average

Token Rules of Thumb

"Hello, how are you?" ≈ 5–6 tokens

A full page (~500 words) ≈ 650–700 tokens

Use these to estimate before you build.



Two Token Counts on Every API Call

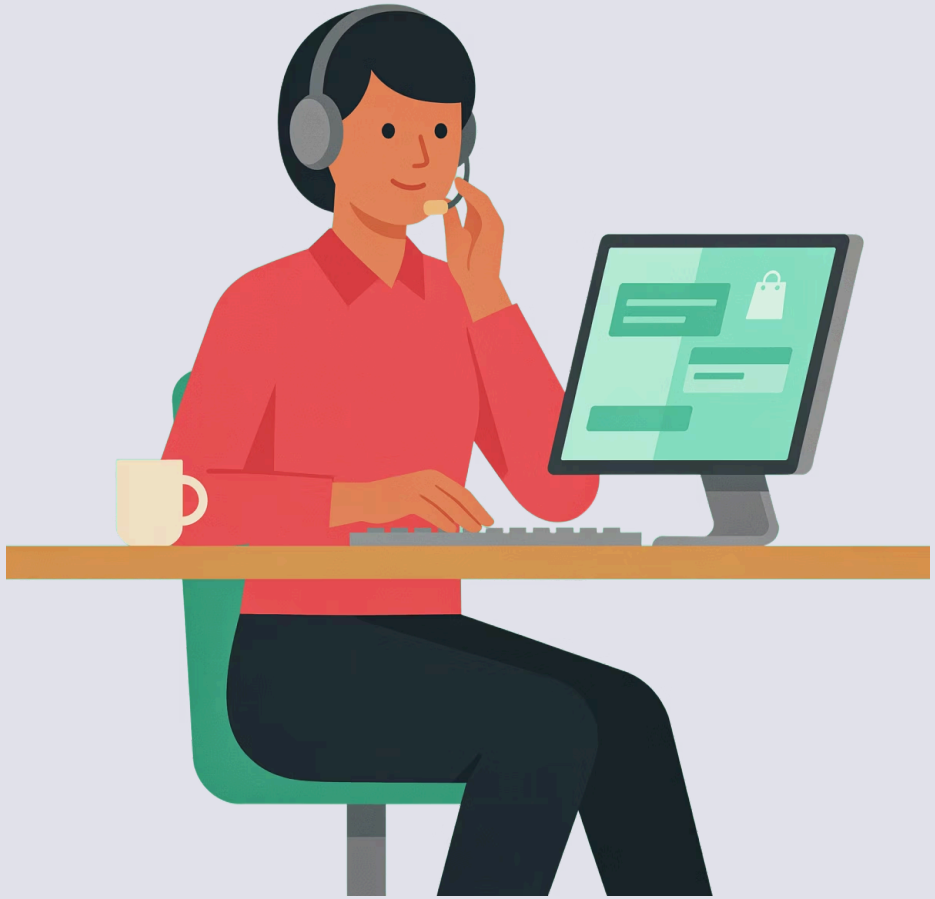
Type	What it means
Input tokens	What you *send*: system prompt + user message + conversation history
Output tokens	What the model *generates* back to you

You are billed for both.

Case Study: Customer Support Chatbot

You are building a support chatbot for an e-commerce store, **ShopEasy**.

Let's cost out a single call.



The Setup: One Call

System prompt (~20 tokens)

"You are a helpful support agent for ShopEasy. Be concise and friendly."

User message (~18 tokens)

"Where is my order #12345? I placed it 3 days ago."

Model response (~30 tokens)

"Hi! Order #12345 was shipped yesterday via FedEx. Tracking number: FX998877. Expected delivery: tomorrow."

Cost Breakdown: Single Call

Component	Tokens	Rate (example)	Cost
Input (prompt)	38	$3 / 1M tokens	$0.000114
Output (response)	30	$15 / 1M tokens	$0.000450
Total	**68**		**$0.000564**

Scale Matters

One call costs a fraction of a cent. But at **100,000 calls per day**:

~$56 per day

~$1,700 per month

Tiny costs multiply fast.



The Danger: Runaway Calls

Costs spiral when:

1. **No max_tokens limit: the model writes a novel instead of a sentence**
2. **Unbounded loops: e.g. a retry loop with a bug keeps calling the API**
3. **History grows untrimmed: every message gets more expensive**
4. **System prompts balloon with unnecessary instructions**



Runaway Cost: Dangerous Code

```
# DANGEROUS: no max_tokens, history grows forever
history = []
while True:
 user_input = input("You: ")
 history.append({"role": "user", "content": user_input})
 response = client.messages.create(
 model="claude-sonnet-4-6",
 messages=history # grows with every turn!
 # max_tokens not set!
 # model can output thousands of tokens
 )
 reply = response.content[0].text
 history.append({"role": "assistant", "content": reply})
 print("Bot:", reply)
```

Why That Code Is Dangerous

After 50 turns, history could contain **10,000+ tokens**.

That entire history is sent on **every single call**, and you pay for it every time.

Best Practice 1: Always Set max_tokens

```
# SAFE: output is capped
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=256,  # <-- hard cap on output length
    messages=[{"role": "user", "content": user_input}]
)
```

A hard cap on output length means no surprise essays.

Choosing a max_tokens Value

Use case	Suggested max_tokens
Short answers / Q&A	128 – 256
Summaries	256 – 512
Code generation	512 – 1024
Long-form writing	1024 – 4096

Best Practice 2: Trim Conversation History

Don't send the whole conversation forever. Use a **sliding window**:

```
# Keep only the last 10 messages
MAX_HISTORY = 10
history.append({"role": "user", "content": user_input})
trimmed_history = history[-MAX_HISTORY:] # sliding window
response = client.messages.create(
 model="claude-sonnet-4-6",
 max_tokens=256,
 messages=trimmed_history
)
```

Best Practice 3: Estimate Cost Before You Run

> Estimated Cost = (Avg Input Tokens × Input Rate) + (Avg Output Tokens × Output Rate)
>
> Then multiply by expected call volume.

Calculate **before** you ship, not after the invoice arrives.

Quick Cost Estimator

```
avg_input_tokens = 500
avg_output_tokens = 200
calls_per_day = 10_000
input_rate = 3.00 / 1_000_000 # $ per token
output_rate = 15.00 / 1_000_000 # $ per token

daily_cost = calls_per_day * (
 avg_input_tokens * input_rate +
 avg_output_tokens * output_rate
)
print(f"Estimated daily cost: ${daily_cost:.2f}")
# → Estimated daily cost: $45.00
```

Best Practice 4: Set Budget Alerts

Most cloud providers and API dashboards offer spending limits and alert thresholds

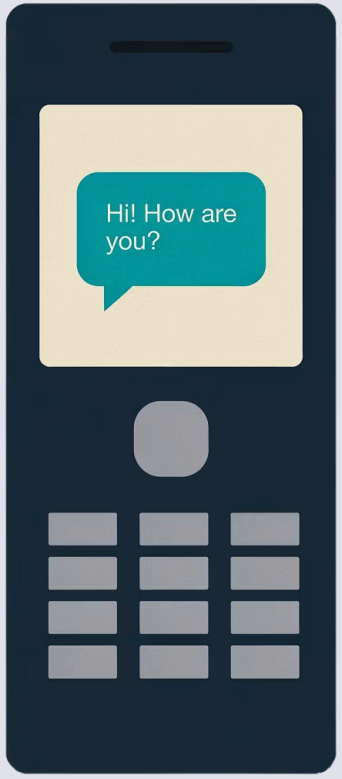
Configure them before deploying to production

Your last line of defence when something else goes wrong



Summary Cheat Sheet

Concept	Key takeaway
Tokens	~4 chars each; you pay for input AND output
max_tokens	Always set it: caps output and prevents surprises
History trimming	Use a sliding window; don't send the whole chat forever
Cost estimation	Calculate before you ship: tokens × rate × volume
Budget alerts	Set them on the API dashboard before going live



Mental Model: Old-School SMS

Think of tokens like SMS characters in the old days:

You paid per character sent and per character received

The longer the message, the higher the bill

Always ask: "Do I really need to send all of this?"

Quick Check: Discuss in Pairs

1. Why can a "cheap" call at $0.0005 still become a big bill?
2. Name three ways a bot could accidentally run up costs.
3. A chat has run for 40 turns. Why is turn 40 more expensive than turn 1?
4. Estimate the daily cost: 2,000 input tokens, 500 output tokens, 5,000 calls per day, at $3 / $15 per 1M tokens.